

Data Transformation Lab: Feature Engineering & Preprocessing

Objective

In this lab, you will explore the third major phase of Data Preprocessing: **Data Transformation**. You will apply various statistical and encoding techniques to transform raw numerical and categorical data into formats optimized for machine learning algorithms. The provided dataset contains 65,000 records of simulated customer data.

Dataset Overview

The dataset (data_transformation_dataset.csv) contains customer transaction profiles. Familiarize yourself with the key features:

Column Name	Variable Type	Description
Age, Income, Credit_Score	Continuous Numerical	Quantitative variables on vastly different numerical scales.
Transaction_Amount	Continuous Numerical	A highly right-skewed variable mimicking real-world transaction data.
Region	Nominal Categorical	Geographical groupings with no inherent ranking.
Education, Subscription_Type	Ordinal Categorical	Categorical labels that possess a natural, logical order.

Lab Tasks

Task 1: Feature Scaling - Standardization (Z-Score Normalization)

Goal: Scale the Income column so that it has a mean of 0 and a standard deviation of 1. This

prevents algorithms like SVMs or Neural Networks from being biased by large magnitudes.

1. Import StandardScaler from sklearn.preprocessing.
2. Fit and transform the Income column.
3. Create a new column named Income_Standardized.
4. Verify your work by printing the new mean (should be ~0) and standard deviation (should be ~1).

Task 2: Feature Scaling - Min-Max Normalization

Goal: Scale the Credit_Score column into a fixed, constrained range between 0 and 1.

1. Import MinMaxScaler from sklearn.preprocessing.
2. Apply the scaler to the Credit_Score column.
3. Create a new column named Credit_Score_Normalized.
4. Confirm the transformation by outputting the min and max values of this new column.

Task 3: One-Hot Encoding (Nominal Data)

Goal: Convert the nominal variable Region into machine-readable binary columns (0s and 1s) without imposing an artificial order.

1. Use pandas.get_dummies() or OneHotEncoder from sklearn.preprocessing on the Region column.
2. **Critical:** Drop the first category (e.g., using drop_first=True in pandas) to avoid the dummy variable trap, which causes multicollinearity.
3. Concatenate the new binary indicator columns to your main dataframe.

Task 4: Label / Ordinal Encoding (Ordinal Data)

Goal: Transform the Education column into numerical representations while preserving its natural rank/order.

1. Identify the correct logical order: High School < Bachelors < Masters < PhD.
2. Use OrdinalEncoder from sklearn.preprocessing, or write a custom mapping dictionary and use pandas' .map() function to assign integer values (e.g., 0, 1, 2, 3) to the categories.
3. Store the resulting integers in a new column called Education_Encoded.

Task 5: Discretization / Binning

Goal: Convert the continuous Age variable into categorical age groups (bins) to help models capture non-linear relationships.

1. Define custom bin edges: [0, 25, 45, 65, 100].

2. Define the corresponding bin labels: 'Youth', 'Adult', 'Middle-Aged', 'Senior'.
3. Use the `pandas.cut()` function to categorize the Age column.
4. Save this to a new column named `Age_Group` and print the `value_counts()` to see the distribution.

Task 6: Log Transformation (Handling Data Skewness)

Goal: Reduce the heavy right-skewness in the `Transaction_Amount` column to force the distribution closer to a normal (Gaussian) curve.

1. Use Matplotlib or Seaborn to plot a histogram of the original `Transaction_Amount`.
Observe the long tail.
2. Apply a mathematical transformation using `numpy.log1p()` (which calculates $\log(1+x)$) to the column to handle potential zeros safely.
3. Store the result in a new column: `Transaction_Amount_Log`.
4. Plot a new histogram of the transformed column to observe the corrected, normalized distribution.

Submission Guidelines

Submit your completed Jupyter Notebook (.ipynb) showcasing the code, textual outputs, and histograms for Task 6. Ensure all newly transformed columns are clearly visible in a final `df.head()` printout.